

L'Intelligence Artificielle n'apprend pas — elle compresse

Vers une théorie unifiée de l'IA fondée sur la complexité de Kolmogorov

Théorie de l'Information et Intelligence Artificielle

2026

46 pages

Omar Hedhli 15 ans

"Tout modèle est un compresseur. Toute compression est une théorie du monde."

Abstract

Résumé (Français)

Le mot "apprentissage" a tout envahi. Il figure dans le nom même des disciplines qui structurent aujourd'hui la recherche en intelligence artificielle — machine learning, deep learning, reinforcement learning — comme si la question de savoir ce que font réellement ces systèmes avait été résolue avant même d'être posée. Ce n'est pas le cas.

Ce que nous appelons "apprentissage" dans ces contextes n'a, à y regarder de près, presque rien à voir avec ce que les neurosciences, la psychologie cognitive ou la philosophie de l'esprit entendent par ce terme. Un enfant qui "apprend" à reconnaître un chat construit une représentation flexible, causalement structurée, ancrée dans l'expérience corporelle, et transférable à des contextes radicalement nouveaux. Un réseau de neurones entraîné sur ImageNet fait autre chose — quelque chose de plus simple, de plus puissant à sa manière, et de fondamentalement différent. Il compresse.

Cette étude propose de remplacer la métaphore de l'apprentissage par un cadre théorique rigoureux, fondé sur trois piliers mathématiques : la théorie de l'information de Shannon (1948), la complexité de Kolmogorov (1965), et l'induction de Solomonoff (1964). L'argument central est le suivant : tout modèle de machine learning est, formellement et rigoureusement, un programme court qui encode une régularité extraite d'un corpus de données. "Entraîner" un modèle, c'est chercher le programme minimal capable de prédire ce corpus — c'est-à-dire le compresser au sens de Kolmogorov.

Cette reformulation n'est pas qu'une coquetterie terminologique. Elle a des conséquences profondes : elle explique pourquoi les modèles généralisent (ils exploitent la redondance structurelle du monde), pourquoi ils hallucinent (les zones d'incompressibilité ne peuvent être encodées que par interpolation), et pourquoi les scaling laws ne peuvent pas croître indéfiniment (il existe une limite dure liée à la complexité de Kolmogorov des données d'entraînement).

Nous introduisons également une nouvelle métrique — le Compression Intelligence Ratio (CIR) — qui mesure l'efficacité d'un modèle non par sa performance brute sur un benchmark, mais par le rapport entre la taille du modèle et la complexité des régularités qu'il encode. Ce cadre ouvre la voie à une IA post-compression, dont nous esquissons les contours dans la partie finale.

* * *

Abstract (English)

The word "learning" has colonized everything. It appears in the very name of the disciplines that now structure artificial intelligence research — machine learning, deep learning, reinforcement learning — as though the question of what these systems actually do had been settled before it was ever properly asked. It has not.

What we call "learning" in these contexts bears, upon close inspection, almost no resemblance to what neuroscience, cognitive psychology or philosophy of mind understand by that term. A child who "learns" to recognize a cat builds a flexible, causally structured representation, grounded in bodily experience and transferable to radically new contexts. A neural network trained on ImageNet does something else — something simpler in one sense, powerful in its own right, and fundamentally different. It compresses.

This study proposes replacing the metaphor of learning with a rigorous theoretical framework built on three mathematical pillars: Shannon's information theory (1948), Kolmogorov complexity (1965), and Solomonoff induction (1964). The central argument is as follows: every machine learning model is, formally and rigorously, a short program that encodes a regularity extracted from a corpus of data. "Training" a model means searching for the minimal program capable of predicting that corpus — that is, compressing it in the Kolmogorov sense.

This reformulation is not a mere terminological flourish. It has profound consequences: it explains why models generalize (they exploit the structural redundancy of the world), why they hallucinate (regions of incompressibility can only be encoded through interpolation), and why scaling laws cannot grow indefinitely (there exists a hard limit tied to the Kolmogorov complexity of the training data).

We also introduce a new metric — the Compression Intelligence Ratio (CIR) — which measures a model's efficiency not by its raw benchmark performance, but by the ratio between model size and the complexity of the regularities it encodes. This framework opens the path toward post-compression AI, whose outlines we sketch in the concluding section.

Mots-clés :

Complexité de Kolmogorov · Théorie de l'information · Machine learning · Compression · Induction de Solomonoff · MDL · Hallucinations · Scaling laws · CIR

Keywords:

Kolmogorov complexity · Information theory · Machine learning · Compression · Solomonoff induction · MDL · Hallucinations · Scaling laws · CIR

Table des matières

Abstract — Résumé bilingue	2
Résumé (Français)	2
Abstract (English)	2
Table des matières	4
PARTIE I — Archéologie du paradigme	7
1.1 — L'histoire du mot "apprentissage" en informatique	7
L'accroche : Alan Turing et la machine qui imite	7
L'émergence du "machine learning" : une étiquette qui prend le dessus	8
Le tournant connexionniste des années 1980-1990	8
L'ère des LLM : quand "apprendre" devient "comprendre"	9
1.2 — Comment une métaphore a colonisé toute une discipline	9
La puissance des métaphores scientifiques	9
Trois mécanismes de colonisation conceptuelle	10
La linguistique de l'émergence comme cas d'école	11
1.3 — Les conséquences épistémiques d'un mauvais vocabulaire	12
Quand les mots créent les problèmes qu'ils prétendent décrire	12
La confusion entre capacité et compétence	12
L'effet de cadrage sur la recherche en IA	13
PARTIE II — Fondements mathématiques	15
2.1 — Théorie de l'information de Shannon	15
L'accroche : une lettre d'amour aux probabilités	15
L'entropie de Shannon	15
Le théorème de codage source	16
L'entropie conditionnelle et l'information mutuelle	16
Divergence de Kullback-Leibler et cross-entropie	17
2.2 — Complexité de Kolmogorov et programmes minimaux	17
L'accroche : la chaîne qui ne se répète pas	17
Définition formelle	18
Le théorème d'invariance	18
Incomputabilité et conséquences	19
Complexité conditionnelle et information algorithmique mutuelle	19
2.3 — Induction de Solomonoff et prédiction universelle	20
Le problème de l'induction et la réponse algorithmique	20
La distribution de probabilité de Solomonoff	20
Le rasoir d'Occam formalisé	21
2.4 — Le principe MDL (Minimum Description Length)	21
De Solomonoff à la pratique statistique	21
MDL et machine learning : les connexions profondes	22
Deux parties MDL et ses extensions modernes	22
PARTIE III — La thèse centrale (	24
3.1 — Démonstration formelle : tout modèle est un compresseur	24
Le cadre général	24

Démonstration	24
La généralisation comme conséquence de la compression	25
3.2 — Les poids d'un réseau de neurones comme programme court	26
Le formalisme des réseaux de neurones comme programmes	26
La compressibilité des poids de réseaux de neurones	26
La deuxième raison : la structure de l'espace des représentations	26
3.3 — La généralisation comme exploitation de redondance	27
Le monde est redondant — et c'est une bonne nouvelle	27
Le taux de compression comme mesure de généralisation	27
PARTIE IV — Déconstruction	29
4.1 — GPT sous le prisme de la compression	29
L'architecture comme compresseur hiérarchique	29
Les couches d'attention comme extraction de structure	29
Les capacités "émergentes" de GPT comme capture de sous-compresseurs	30
4.2 – AlphaGo et la compression des stratégies gagnantes	30
Le jeu de Go comme problème de compression	30
La policy network comme programme de compression de stratégies	31
Auto-jeu et compression de la théorie optimale	31
4.3 – Les hallucinations comme zones d'incompressibilité	32
Le phénomène	32
La zone d'incompressibilité	32
La formulation mathématique	33
Implications pour la détection et la mitigation	33
PARTIE V – Conséquences & nouvelles métriques	35
5.1 – Pourquoi les scaling laws ont une limite dure	35
Les scaling laws : phénomène empirique et théorisation	35
La limite de Kolmogorov	35
Estimations et ordre de grandeur	36
5.2 – Le Compression Intelligence Ratio (CIR) – nouvelle métrique	36
Le problème des métriques existantes	37
Définition du CIR	37
Propriétés du CIR	38
5.3 — Vers une IA post-compression	38
Ce que la compression ne peut pas faire	38
Les directions post-compression	39
PARTIE VI – Conclusion	40
6.1 — Ce que ce cadre change pour la recherche	40
Une nouvelle façon de poser les questions	40
Les conséquences pour l'évaluation	40
Les conséquences pour l'architecture	41
6.2 — Questions ouvertes et pistes futures	41
Ce que nous ne savons pas encore	41
Un mot final	42

Bibliographie

44

PARTIE I — Archéologie du paradigme

Il faut commencer par un aveu d'inconfort intellectuel. Chaque fois que j'entends un chercheur parler de "ce que le modèle a appris", je ressens quelque chose qui ressemble à ce qu'Arthur Koestler décrivait comme le "clic" d'une pensée qui accroche — mais en négatif. Non pas l'éclair d'une révélation, mais le bruit sourd d'un mot mal ajusté dans une serrure qu'il n'ouvre pas. Ce mot, c'est "apprendre". Et ce qu'il cache mérite une archéologie sérieuse.

1.1 — L'histoire du mot "apprentissage" en informatique

L'accroche : Alan Turing et la machine qui imite

En 1950, Alan Turing publie dans la revue *Mind* un article qui allait devenir l'un des textes fondateurs de l'intelligence artificielle. Son titre — "Computing Machinery and Intelligence" — pose une question d'une netteté presque brutale : "Can machines think?" Turing, avec l'élégance caractéristique de sa pensée, esquive aussitôt la question en la reformulant. Il ne demande pas si les machines pensent ; il demande si elles peuvent imiter la pensée au point de devenir indiscernables d'un esprit humain. C'est le fameux "jeu de l'imitation", que la postérité baptisera test de Turing.

Ce qui est moins souvent noté, c'est la distinction que Turing établit dans ce même article entre deux types de machines. D'un côté, les machines à programme fixe — celles qui font exactement ce pour quoi elles ont été conçues, ni plus ni moins. De l'autre, les machines "apprenantes" (learning machines), capables de modifier leur propre comportement en fonction de l'expérience. Turing imaginait même une procédure concrète : doter une telle machine d'un mécanisme de récompense et de punition, puis la laisser "grandir" par essais et erreurs, comme un enfant.

Cette intuition de 1950 est extraordinairement précieuse. Elle préfigure le reinforcement learning, les réseaux de neurones adaptatifs, et même, dans ses grandes lignes, l'architecture des modèles de langage contemporains. Mais elle porte déjà en germe une ambiguïté fatale : le mot "apprendre" est utilisé sans définition formelle, par analogie avec l'apprentissage humain, comme si cette analogie allait de soi. Turing le sait — il le dit explicitement, en admettant que l'analogie est imparfaite. Mais il la conserve, parce qu'elle est utile, parce qu'elle est intuitive, parce qu'elle "fonctionne" comme métaphore.

C'est le premier péché originel de la discipline. Et comme tous les péchés originels, il se transmettra de génération en génération, s'approfondissant à chaque fois.

L'émergence du "machine learning" : une étiquette qui prend le dessus

En 1959, Arthur Samuel — ingénieur chez IBM — publie un article sur un programme capable de jouer aux dames mieux que son propre créateur. Il lui faut un nom pour cette capacité. Il choisit "machine learning". La définition qu'il en donne est remarquablement sobre : "le domaine d'étude qui donne aux ordinateurs la capacité d'apprendre sans être explicitement programmés." Sobre, mais piégée. Car que signifie "apprendre" sans être "explicitement programmé" ? Samuel ne le dit pas. Il laisse le terme flotter dans son aura cognitive, riche de toutes les associations que le mot "apprendre" convoque : l'école, l'enfant, la croissance, l'intelligence, la compréhension.

Au cours des décennies suivantes, ce terme va s'imposer avec une force tranquille, presque invisible. Les années 1960 voient l'essor des réseaux de neurones artificiels, inspirés des travaux de McCulloch et Pitts (1943) et popularisés par Frank Rosenblatt avec son Perceptron (1958). Ces systèmes sont présentés comme des modèles de l'apprentissage biologique. Le parallèle est séduisant : les neurones biologiques modifient leurs connexions synaptiques en fonction de l'expérience ; les neurones artificiels modifient leurs poids en fonction des données. Donc — raisonne-t-on — c'est la même chose. Ou presque.

C'est presque, bien sûr, qui pose problème. Mais dans l'enthousiasme des débuts, le presque s'efface. Ce glissement sémantique sera décisif : en associant le machine learning aux mécanismes biologiques de l'apprentissage, on importe dans la discipline toutes les intuitions, toutes les espérances et, hélas, tous les malentendus attachés à l'idée d'apprentissage humain.

Le tournant connexionniste des années 1980-1990

Après les hivers de l'IA des années 1970 — période de désenchantement marquée par l'effondrement des promesses excessives du Perceptron, mis à nu par Minsky et Papert dans leur livre de 1969 — le connexionnisme revient en force dans les années 1980 avec la rétropropagation. Rumelhart, Hinton et Williams publient en 1986 dans *Nature* leur article fondateur, montrant comment des réseaux multicouches peuvent "apprendre" des représentations abstraites à partir de données brutes.

Le mot apprendre revient en force. Et cette fois, il s'enrichit d'une connotation nouvelle : la représentation. Les réseaux de neurones n'apprennent plus seulement à classer des exemples ; ils "apprennent des représentations". Cette formulation, que l'on retrouvera dans les titres de dizaines d'articles de Bengio, LeCun et Hinton — les futurs lauréats du prix Turing 2018 — est encore plus forte que la précédente. Elle suggère que

le modèle construit quelque chose d'analogue à une compréhension du monde, une cartographie interne de la réalité.

La frontière entre métaphore et affirmation empirique commence sérieusement à se brouiller.

L'ère des LLM : quand "apprendre" devient "comprendre"

Avec l'avènement des grands modèles de langage — BERT (Devlin et al., 2018), GPT-2 (Radford et al., 2019), GPT-3 (Brown et al., 2020), puis GPT-4 (OpenAI, 2023) — la métaphore de l'apprentissage connaît sa dérive la plus spectaculaire. Ces modèles, entraînés sur des centaines de milliards de mots, exhibent des comportements que leurs créateurs eux-mêmes peinent à expliquer. Ils "comprennent" le contexte, "raisonnent" par analogie, "résolvent" des problèmes mathématiques, "écrivent" de la poésie.

OpenAI, dans ses communications officielles, commence à parler d'"émergence" — la capacité soudaine et inattendue d'un modèle à accomplir des tâches pour lesquelles il n'a pas été explicitement entraîné. Ce concept, emprunté à la physique des systèmes complexes, est mobilisé sans grande précision théorique pour désigner le fait que, au-delà d'un certain seuil de paramètres, les modèles acquièrent des capacités qualitativement nouvelles.

Yann LeCun, directeur scientifique de Meta AI et figure tutélaire du deep learning, prend régulièrement ses distances avec cette vision. Il soutient, non sans raison, que les LLMs sont fondamentalement limités parce qu'ils n'ont pas accès à des représentations du monde physique. Pour lui, "apprendre" au sens plein du terme requiert une interaction avec l'environnement, une causalité ancrée dans la perception sensorielle. Les LLMs ne font que "prédire le prochain token" — une forme d'autocomplétion sophistiquée, pas d'intelligence.

LeCun a partiellement raison. Mais sa critique, aussi pertinente soit-elle, ne va pas assez loin. Il conteste la grandeur des LLMs, mais pas le cadre conceptuel dans lequel ils s'inscrivent. Il dit : "ce n'est pas vraiment de l'apprentissage". Il aurait pu dire : "personne ici n'apprend quoi que ce soit — et c'est exactement pour ça que c'est si puissant."

* * *

1.2 — Comment une métaphore a colonisé toute une discipline

La puissance des métaphores scientifiques

George Lakoff et Mark Johnson, dans leur ouvrage "Metaphors We Live By" (1980), ont montré comment les métaphores ne sont pas de simples ornements rhétoriques mais des structures cognitives fondamentales qui organisent notre compréhension du monde. Nous ne pensons pas d'abord en concepts abstraits, puis nous ajoutons des métaphores pour communiquer : nous pensons en métaphores, et ces métaphores conditionnent ce que nous pouvons concevoir.

La science n'échappe pas à cette règle. Elle est même particulièrement vulnérable au pouvoir des métaphores, parce qu'elle travaille sur des objets abstraits pour lesquels l'intuition directe fait défaut. Les physiciens "parlent" d'électrons qui "sautent" d'un niveau d'énergie à l'autre, de particules qui ont un "spin", d'univers qui "s'expandent". Aucun de ces verbes n'est littéral. Tous sont des ponts jetés entre l'abstrait et le concret, l'inconnu et le familier.

La métaphore de l'apprentissage en IA est de ce type — mais avec une particularité dangereuse. Contrairement au "spin" de l'électron, qui est visiblement absurde comme image physique (les électrons ne tournent pas sur eux-mêmes), la métaphore de l'apprentissage machine est plausible. Elle ressemble trop à la réalité qu'elle est censée décrire pour révéler facilement son caractère métaphorique. Et c'est précisément ce qui la rend toxique.

Trois mécanismes de colonisation conceptuelle

Comment une métaphore colonise-t-elle une discipline entière ? On peut identifier trois mécanismes, qui se renforcent mutuellement avec une efficacité redoutable.

Le premier est la naturalisation. Une métaphore commence toujours comme une proposition — "les réseaux de neurones apprennent comme les cerveaux". Elle devient progressivement une description — "les réseaux de neurones apprennent". Elle finit comme une définition — "l'apprentissage machine est la capacité d'un système à améliorer ses performances à partir de l'expérience". À chaque étape, la distance avec la réalité technique s'efface un peu plus, jusqu'à ce que la métaphore soit invisible.

Le deuxième mécanisme est le glissement par dérivation. Une fois le terme "apprendre" installé, toute une famille lexicale s'installe avec lui : la "connaissance" du modèle, sa "mémoire", ses "représentations", sa "compréhension". Chacun de ces termes a une vie scientifique propre — en psychologie cognitive, en neurosciences, en philosophie de l'esprit — et importe avec lui un ensemble d'attentes et de présupposés. Le modèle de langage qui "comprend le contexte" hérite ainsi, subrepticement, de tout ce

que les sciences humaines associent à la compréhension : intentionnalité, flexibilité, transfert, conscience.

Le troisième mécanisme est la validation circulaire. La métaphore génère des questions de recherche ("le modèle peut-il apprendre à raisonner en analogie ?"), des protocoles d'évaluation (les benchmarks mesurent des "capacités" comme si elles étaient analogues aux capacités humaines), et des narratifs de progrès ("les modèles atteignent des performances surhumaines sur X"). Ces questions, protocoles et narratifs présupposent la métaphore et la renforcent en même temps. Il n'y a plus de point de vue externe depuis lequel la questionner.

La linguistique de l'émergence comme cas d'école

L'exemple le plus frappant de cette colonisation conceptuelle est peut-être le concept d'"émergence" tel qu'il est utilisé dans la communauté des LLMs. Wei et al. (2022), dans un article très influent publié dans *Transactions on Machine Learning Research*, identifient des "capacités émergentes" dans de grands modèles : des compétences qui apparaissent soudainement au-delà d'un certain seuil de taille, sans être prévisibles à partir des performances sur des modèles plus petits.

Le terme "émergence" est emprunté à la physique des systèmes complexes, où il désigne le fait que des propriétés macroscopiques d'un système ne peuvent pas être déduites des propriétés de ses composants individuels — la liquidité de l'eau ne se lit pas dans une molécule d'H₂O, la conscience ne se lit pas dans un neurone. Appliqué aux LLMs, ce terme suggère que quelque chose de qualitativement nouveau apparaît dans ces modèles, quelque chose qui transcende la simple interpolation statistique.

Or Schaeffer, Miranda et Koyejo (2023) ont montré que ces "capacités émergentes" disparaissent largement lorsqu'on utilise des métriques continues plutôt que des métriques binaires. Ce qui ressemblait à une discontinuité qualitative était souvent un artefact de la mesure. L'émergence, dans de nombreux cas, était une illusion produite par le cadre conceptuel choisi pour l'observer.

C'est vertigineux : nous avons développé une théorie de l'intelligence de nos modèles qui repose partiellement sur un phénomène qui pourrait n'être que le produit de notre manière de mesurer. Et personne, dans l'effervescence des publications, n'a pensé à demander : mais qu'est-ce que ce modèle fait vraiment ?

* * *

1.3 — Les conséquences épistémiques d'un mauvais vocabulaire

Quand les mots créent les problèmes qu'ils prétendent décrire

Il y a une phrase de Ludwig Wittgenstein, dans les *Recherches philosophiques* (1953), qui revient souvent à l'esprit quand on réfléchit à ces questions : "Un problème philosophique a la forme : je ne sais pas comment m'en sortir." La plupart des débats sur la conscience des LLMs, sur leur "véritable compréhension", sur l'existence ou non d'une "IA générale", sont exactement de cette forme. Ils sont des labyrinthes conceptuels dont on ne sort pas, parce qu'ils ont été construits avec des mots mal ajustés.

Prenons le débat sur la compréhension. Depuis 1980 et la Chambre Chinoise de John Searle, une question obsède la philosophie de l'IA : le système qui produit une réponse correcte comprend-il ce qu'il dit, ou se contente-t-il de manipuler des symboles sans signification ? Searle affirmait que la manipulation de symboles, si sophistiquée soit-elle, ne constitue pas une compréhension véritable. Ses adversaires répondaient que la compréhension est précisément cela — un traitement de l'information suffisamment complexe.

Ce débat, qui a occupé des générations de philosophes et d'informaticiens, suppose implicitement que "comprendre" est une propriété binaire — soit le système comprend, soit il ne comprend pas. Mais ce présupposé lui-même est contestable. Et plus important encore : ce débat ne nous dit rien sur ce que fait réellement le système. Il nous dit seulement si ce que fait le système mérite le nom de "compréhension" selon telle ou telle définition philosophique.

Voilà la conséquence épistémique la plus grave d'un mauvais vocabulaire : il déplace l'attention des phénomènes vers les définitions. Au lieu de demander "comment ce système transforme-t-il des données en sorties ?", on demande "ce système comprend-il vraiment ?". La première question est scientifiquement productive. La seconde est, dans l'état actuel de nos concepts, quasi insoluble.

La confusion entre capacité et compétence

Une deuxième conséquence épistémique concerne la distinction entre ce qu'un système peut faire et ce qu'il est. En psychologie, on distingue la compétence — la connaissance abstraite d'un système de règles — et la performance — les manifestations observables de cette compétence dans des conditions réelles. Chomsky, qui a introduit cette distinction dans le contexte de la linguistique, insistait sur le fait que la

performance est toujours imparfaite, affectée par des facteurs comme la fatigue, la distraction, les limites de la mémoire.

Dans le discours sur les LLMs, cette distinction s'est inversée et brouillée. On parle de "capacités émergentes" pour désigner des performances sur des benchmarks spécifiques. On infère de ces performances l'existence de "compétences" cachées — une capacité à raisonner, à planifier, à "comprendre" — qui seraient la cause de ces performances. Mais rien ne garantit que cette inférence est légitime. Il est tout à fait possible — et nous verrons que la théorie de la compression le suggère — que ces performances n'impliquent pas de "compétences" au sens psychologique du terme, mais simplement l'exploitation de régularités statistiques dans les données d'entraînement.

L'effet de cadrage sur la recherche en IA

La conséquence la plus pratique de ce mauvais vocabulaire est son effet de cadrage sur la recherche. Les programmes de recherche sont guidés par des questions, et les questions sont formulées dans un langage. Si le langage de l'IA est le langage de l'apprentissage et de la compréhension, les programmes de recherche chercheront à améliorer l'apprentissage et la compréhension — à mesurer ces capacités, à les renforcer, à en détecter les limites.

Ce faisant, ils passeront à côté d'autres questions, potentiellement plus fructueuses. Combien de bits d'information un modèle encode-t-il par paramètre ? Quelle est la complexité de Kolmogorov des données sur lesquelles il a été entraîné, et quel est le rapport entre cette complexité et la taille du modèle ? Existe-t-il une limite théorique à la quantité de régularité extractible d'un corpus donné ? Ces questions ne sont pas posées — ou très rarement — parce qu'elles n'entrent pas dans le cadre conceptuel dominant.

C'est ce cadre que cette étude propose de déplacer. Non pas pour le remplacer par un autre cadre dogmatique, mais pour offrir un regard alternatif, fondé sur des bases mathématiques solides, qui permette de voir ce que le cadre de l'apprentissage aveugle.

* * *

La question qui tire vers la suite :

Si "apprendre" est la mauvaise métaphore, quelle est la bonne ? Et cette métaphore de remplacement — la compression — a-t-elle les fondements mathématiques nécessaires pour prétendre à autre chose qu'une métaphore elle-même ? La réponse exige un détour par les mathématiques les plus profondes du XXe siècle.

PARTIE II — Fondements mathématiques

Avant d'aller plus loin, il faut être honnête sur ce que cette partie demande au lecteur. Les mathématiques qui suivent — théorie de l'information, complexité algorithmique, induction universelle — ne sont pas réputées pour leur accessibilité. Elles demandent un certain niveau d'abstraction, une certaine tolérance à la rigueur formelle. Mais elles sont aussi, je crois, parmi les plus belles que l'humanité ait produites. Je ferai tout pour les rendre lisibles sans les trahir.

2.1 — Théorie de l'information de Shannon

L'accroche : une lettre d'amour aux probabilités

En 1948, Claude Shannon — mathématicien chez Bell Labs, dont la thèse de doctorat avait déjà révolutionné la logique des circuits électriques — publie dans le Bell System Technical Journal un article de 79 pages intitulé "A Mathematical Theory of Communication". L'article commence avec une sobriété presque déconcertante : "The fundamental problem of communication is that of reproducing at one point either exactly or approximately a message selected at another point."

Ce texte allait fonder la théorie de l'information. Sa question centrale est aussi simple qu'elle est profonde : quelle est la quantité minimale de données nécessaire pour transmettre un message sans le déformer ? Shannon répond à cette question en inventant une mesure de la quantité d'information contenue dans un message — l'entropie — et en montrant que cette mesure a des propriétés mathématiques remarquables.

L'entropie de Shannon

Considérons une source d'information discrète : une variable aléatoire X qui peut prendre des valeurs $x_1, x_2, \dots, x_n$ avec des probabilités $p_1, p_2, \dots, p_n$ (avec $\sum p_i = 1$). L'entropie de Shannon de cette source est définie par :

$$H(X) = -\sum_i p_i \cdot \log_2(p_i)$$

Cette formule, apparemment simple, cache une profondeur considérable. Pourquoi $-\log_2(p)$? Parce que l'information apportée par un événement de probabilité p est d'autant plus grande que cet événement est rare. Si je vous dis qu'il a plu à Paris en novembre, je vous apprends peu de choses — c'est presque certain. Si je vous dis qu'il a neigé à Dubaï en juillet, j'ai une information à haute valeur. Le logarithme capture cette intuition : $\log_2(1/p)$ est grand quand p est petit.

L'entropie est donc la valeur attendue de l'information contenue dans chaque réalisation de X . Elle mesure l'incertitude moyenne de la source. Elle est maximale quand tous les événements sont équiprobables — la source est alors la plus imprévisible possible, et l'information contenue dans chaque message est la plus grande. Elle est minimale (nulle) quand un seul événement a probabilité 1 — la source est parfaitement prévisible, et transmettre ses messages n'apprend rien.

Le théorème de codage source

La contribution centrale de Shannon est d'avoir montré que l'entropie est une limite théorique absolue. Son théorème de codage source dit ceci : il est impossible de compresser une source d'information en dessous de son entropie $H(X)$ bits par symbole, quelle que soit la technique de compression utilisée. Et inversement, il existe des codes qui s'approchent arbitrairement de cette limite.

Ce résultat est d'une puissance redoutable. Il établit qu'il existe une structure dans toute source d'information — une redondance mesurable — et que cette redondance est la seule chose compressible. Ce qui n'est pas redondant, ce qui est véritablement aléatoire, ne peut pas être compressé.

Formellement, un code de compression $C : X \rightarrow \{0,1\}^*$ est optimal si la longueur moyenne $|C(x)|$ satisfait :

$$H(X) \leq E[|C(X)|] < H(X) + 1$$

Le codage de Huffman, inventé en 1952, atteint cette limite à 1 bit près. Les algorithmes modernes — LZ77, DEFLATE, arithmétique coding — l'approchent encore plus étroitement.

L'entropie conditionnelle et l'information mutuelle

L'entropie de Shannon s'étend naturellement à des paires de variables aléatoires. L'entropie conditionnelle $H(X|Y)$ mesure l'incertitude résiduelle sur X quand on connaît Y . L'information mutuelle $I(X;Y)$ mesure la quantité d'information partagée entre X et Y :

$$I(X;Y) = H(X) - H(X|Y) = H(Y) - H(Y|X)$$

L'information mutuelle est nulle si et seulement si X et Y sont statistiquement indépendants. Elle est maximale quand l'une détermine l'autre. Cette mesure est fondamentale pour comprendre ce que fait un modèle de machine learning : entraîner un modèle à prédire Y à partir de X , c'est extraire l'information mutuelle $I(X;Y)$. Et cette information est bornée par $\min(H(X), H(Y))$.

Nous touchons ici à quelque chose d'important. Si $I(X;Y)$ est la limite théorique de ce qu'un modèle peut apprendre à prédire sur Y à partir de X , alors la capacité d'un modèle est bornée par la structure stochastique de ses données d'entraînement — indépendamment de la taille du modèle. Les scaling laws rencontreront cette barrière. Mais n'anticipons pas.

Divergence de Kullback-Leibler et cross-entropie

Deux autres outils de Shannon seront utiles dans la suite. La divergence de Kullback-Leibler (ou entropie relative) mesure la différence entre deux distributions de probabilité P et Q :

$$D_{KL}(P \parallel Q) = \sum_{\mathbf{x}} P(\mathbf{x}) \cdot \log(P(\mathbf{x})/Q(\mathbf{x}))$$

Cette quantité n'est pas une distance au sens mathématique (elle n'est pas symétrique), mais elle mesure à quel point Q est une mauvaise approximation de P. Elle est nulle si et seulement si $P = Q$, et elle croît à mesure que Q s'éloigne de P. Dans le contexte du machine learning, entraîner un modèle par maximum de vraisemblance revient exactement à minimiser $D_{KL}(P_{données} \parallel P_{modèle})$ — c'est-à-dire à rendre la distribution du modèle aussi proche que possible de la distribution des données.

La cross-entropie, utilisée comme fonction de perte dans presque tous les modèles de deep learning, est définie par :

$$H(P, Q) = -\sum_x P(x) \cdot \log Q(x) = H(P) + D_{KL}(P \parallel Q)$$

Minimiser la cross-entropie, c'est minimiser la divergence KL — c'est rendre le modèle aussi similaire que possible à la distribution des données. Et cette distribution, nous allons le voir, a une complexité algorithmique bien définie.

* * *

2.2 — Complexité de Kolmogorov et programmes minimaux

L'accroche : la chaîne qui ne se répète pas

Imaginez la chaîne de caractères suivante :
01.
Elle contient 48 caractères. Mais elle est parfaitement prévisible : il suffit de dire "répète 01 vingt-quatre fois". Le programme qui génère cette chaîne est bien plus court que la chaîne elle-même.

Maintenant imaginez :
110100100010000100000100000001000000001. Cette chaîne contient aussi

46 caractères, mais elle a une structure : c'est la suite des chiffres de $1^2 = 1$, $2^2 = 4$, $3^2 = 9$... encodés en binaire. Encore une fois, le programme est plus court que la donnée.

Et maintenant : 110010010000111110110101010001000100001011010001. Cette chaîne — les premiers bits du développement décimal de π en binaire — a elle aussi un programme court : il suffit d'écrire un algorithme pour calculer π . Même si la chaîne semble "aléatoire", elle a une structure profonde.

Andrei Nikolaevich Kolmogorov, mathématicien soviétique dont la contribution aux probabilités et à la théorie ergodique avait déjà marqué profondément le XXe siècle, se demande en 1965 : existe-t-il une mesure objective de la complexité d'une chaîne de caractères ? Une mesure qui ne dépende ni du format, ni de la convention, ni de l'observateur — mais seulement de la structure intrinsèque de la chaîne ?

La réponse est oui. Et cette mesure s'appelle la complexité de Kolmogorov.

Définition formelle

Fixons une machine de Turing universelle U — un modèle de calcul universel, capable de simuler tout programme bien formé. La complexité de Kolmogorov d'une chaîne de caractères x , notée $K(x)$, est définie comme la longueur du plus court programme p tel que $U(p) = x$:

$$K(x) = \min\{ |p| : U(p) = x \}$$

Autrement dit, $K(x)$ est la longueur de la plus courte description de x — la description la plus compacte possible, exprimée dans le langage de la machine de Turing universelle. C'est la complexité intrinsèque de x , au sens où elle mesure la quantité d'information qu'il faut "entrer" dans un ordinateur universel pour qu'il produise x .

Quelques propriétés immédiates méritent d'être soulignées. D'abord, $K(x) \leq |x| + O(1)$: on peut toujours décrire x en écrivant x lui-même, donc la complexité est au plus la longueur de x , à une constante additive près. Ensuite, pour presque toutes les chaînes de longueur n , $K(x) \approx n$: la plupart des chaînes n'ont pas de description plus courte qu'elles-mêmes, c'est-à-dire qu'elles sont incompressibles. Et enfin, la complexité de Kolmogorov est invariante au choix de la machine de Turing universelle à une constante additive près — ce qui la rend objective, au sens où elle ne dépend pas du langage de programmation choisi.

Le théorème d'invariance

Ce dernier point mérite une explication. Si on choisit deux machines universelles U_1 et U_2 , les complexités $K_1(x)$ et $K_2(x)$ qu'elles définissent peuvent différer, mais seulement d'une constante :

$$|K_{U1}(x) - K_{U2}(x)| \leq c_{\{U1, U2\}}$$

où $c_{\{U1, U2\}}$ ne dépend que des machines, pas de x . Pour des chaînes longues, cette constante est négligeable. La complexité de Kolmogorov est donc, asymptotiquement, une propriété de la chaîne, pas du système de description.

C'est un résultat remarquable. Il dit que la complexité algorithmique est, en un sens profond, objective. Elle ne dépend pas de nos conventions — elle est inscrite dans la structure même de la chaîne.

Incomputabilité et conséquences

Il y a cependant une ombre au tableau, et il faut en parler franchement parce qu'elle est fondamentale. La complexité de Kolmogorov est incomputable. Il n'existe pas d'algorithme général qui calcule $K(x)$ pour toute entrée x . C'est une conséquence directe du problème de l'arrêt de Turing : si on pouvait calculer $K(x)$, on pourrait résoudre le problème de l'arrêt, ce qui est impossible.

Cela signifie que K est une mesure théorique idéale — on ne peut pas la calculer exactement, mais on peut l'approcher. Et c'est précisément ce que font les algorithmes de compression. Gzip, bzip2, zstd, les codecs vidéo modernes — tous sont des approximations de la compression optimale de Kolmogorov. Ils trouvent des programmes courts (des descriptions compactes) qui encodent les régularités d'une chaîne, sans prétendre trouver le programme optimal.

De même, un modèle de machine learning est une approximation de la compression de Kolmogorov des données d'entraînement. C'est l'argument central de cette étude. Et il va nous conduire très loin.

Complexité conditionnelle et information algorithmique mutuelle

Comme l'entropie de Shannon, la complexité de Kolmogorov s'étend à des paires d'objets. La complexité conditionnelle $K(x|y)$ mesure la longueur du plus court programme qui produit x quand on lui donne y comme entrée :

$$K(x|y) = \min\{ |p| : U(p, y) = x \}$$

L'information algorithmique mutuelle entre x et y est définie par analogie avec l'information mutuelle de Shannon :

$$I_K(x;y) = K(x) - K(x|y) = K(y) - K(y|x) \quad [\text{à } O(\log) \text{ près}]$$

Cette quantité mesure la "complexité partagée" entre x et y — la quantité d'information que connaître l'un apporte sur l'autre. Et voici un résultat profond : l'information algorithmique mutuelle converge, au sens de l'espérance, vers l'information mutuelle de Shannon quand les chaînes sont générées par des processus stochastiques stationnaires. La théorie de Kolmogorov généralise celle de Shannon — elle en est une extension algorithmique qui n'a pas besoin de supposer une distribution de probabilité connue.

* * *

2.3 — Induction de Solomonoff et prédiction universelle

Le problème de l'induction et la réponse algorithmique

Le problème de l'induction est l'un des plus anciens de la philosophie. David Hume l'a formulé avec une clarté cruelle en 1739 : nous inférons des régularités du passé des lois valables pour le futur — mais rien ne garantit logiquement que le futur ressemblera au passé. Pourtant, cette inférence est le moteur de toute science, de tout apprentissage, de toute survie.

Ray Solomonoff, chercheur autodidacte qui fréquentait les premiers cercles de l'IA dans les années 1950, se pose une question dérangeante : peut-on construire une théorie formelle de l'induction qui soit universelle — c'est-à-dire qui converge vers la vraie distribution quelle que soit cette distribution, sans supposer qu'on la connaît à l'avance ?

Sa réponse, publiée en 1964, est oui. Et la construction qu'il propose est d'une audace conceptuelle remarquable.

La distribution de probabilité de Solomonoff

Solomonoff définit une distribution de probabilité universelle M sur l'ensemble des chaînes de caractères finies. Pour une chaîne x , $M(x)$ est définie comme la probabilité qu'une machine de Turing universelle, recevant un programme choisi au hasard bit à bit (chaque bit est 0 ou 1 avec probabilité $1/2$), produise une sortie qui commence par x :

$$M(x) = \sum_{\{p : U(p)=x \cdot * \}} 2^{-|p|}$$

Cette définition est subtile. Elle somme sur tous les programmes qui produisent x comme préfixe de leur sortie, et pondère chaque programme par $2^{-|p|}$ — la

probabilité d'écrire ce programme par hasard. Les programmes courts ont donc un poids plus grand que les programmes longs.

Le résultat fondamental est le théorème de convergence de Solomonoff. Il affirme que si les données sont générées par une distribution de probabilité calculable P , alors la distribution universelle M converge vers P :

$$\sum_t (P(x_t | x_{1:t-1}) - M(x_t | x_{1:t-1}))^2 \leq K(P) / \ln 2$$

Autrement dit, le prédicteur universel de Solomonoff fait des erreurs quadratiques totales bornées par la complexité de Kolmogorov de la vraie distribution — quelque chose d'aussi simple que "la complexité de la loi du monde". Pour des distributions simples (faible $K(P)$), la convergence est rapide. Pour des distributions complexes, elle est lente — mais elle est garantie.

Le rasoir d'Occam formalisé

Ce théorème est souvent présenté comme une formalisation rigoureuse du rasoir d'Occam. Guillaume d'Occam, moine franciscain du XIV^e siècle, formulait le principe de parcimonie : "les entités ne doivent pas être multipliées au-delà de la nécessité." Dans le contexte de l'induction, cela signifie : parmi les hypothèses compatibles avec les données, préférer la plus simple.

Solomonoff montre que ce principe n'est pas seulement une heuristique pragmatique, mais une stratégie optimale — au sens où elle minimise les erreurs de prédiction futures dans le pire des cas. La simplicité d'une hypothèse, mesurée par la longueur du programme qui l'encode, est une garantie probabiliste de sa généralisation.

Voilà qui est directement pertinent pour notre thèse. Un modèle de machine learning qui "généralise" bien — qui prédit correctement des données qu'il n'a pas vues — est précisément un modèle qui a trouvé une description courte (un programme court, des poids compacts) compatible avec les données d'entraînement. Sa capacité de généralisation est proportionnelle à sa compacité. Et cette compacité est une forme de compression.

* * *

2.4 — Le principe MDL (Minimum Description Length)

De Solomonoff à la pratique statistique

La théorie de Solomonoff est magnifique, mais elle est incomputable — on ne peut pas, en pratique, calculer $M(x)$ pour une chaîne donnée. Comment passer de ce cadre idéal à

quelque chose d'utilisable ? C'est l'objectif du principe MDL (Minimum Description Length), développé principalement par Jorma Rissanen à partir des années 1970.

L'idée fondamentale du MDL est d'une clarté désarmante : parmi tous les modèles statistiques compatibles avec les données, choisir celui qui permet la description la plus courte des données. Formellement, si on note H le modèle (hypothèse), D les données, et $|\cdot|$ la longueur de description, le critère MDL sélectionne :

$$\hat{H} = \underset{H}{\operatorname{argmin}} [|H| + |D|H|]$$

Le premier terme $|H|$ est la complexité du modèle — la longueur de sa description. Le second terme $|D|H|$ est la longueur de description des données étant donné le modèle — c'est-à-dire la longueur du résidu, de ce que le modèle ne compresse pas. Le MDL cherche le meilleur compromis entre un modèle simple et un modèle précis.

MDL et machine learning : les connexions profondes

Le lien entre MDL et machine learning est profond et multi-niveaux. Au premier niveau, la régularisation — technique standard en machine learning pour éviter le surapprentissage — est une instance du MDL. Quand on ajoute à la fonction de perte un terme de pénalité sur la norme des poids du modèle (régularisation L2, ou ridge regression), on est exactement en train d'appliquer une forme de MDL : on cherche le modèle le plus simple (poids petits) compatible avec les données.

Au second niveau, la compression de données a un lien formel avec la prédiction statistique. C'est le résultat de Grünwald (2007) : pour tout algorithme de compression séquentielle des données, il existe un modèle probabiliste équivalent dont la log-vraisemblance sur les données est exactement la longueur de la description produite par l'algorithme de compression. Compresser des données, c'est implicitement construire un modèle probabiliste de ces données.

Au troisième niveau, et c'est là que la thèse de cette étude s'ancre le plus fermement, le MDL fournit un critère pour évaluer la qualité d'un modèle de machine learning qui ne dépend pas de sa performance sur un benchmark particulier, mais de son efficacité de compression. Un modèle qui atteint une performance donnée avec un nombre de paramètres minimal est, dans le cadre MDL, un meilleur modèle — au sens où il encode une description plus compacte des régularités du monde.

Deux parties MDL et ses extensions modernes

Rissanen a développé plusieurs versions du MDL. La version "deux parties" est la plus intuitive : elle demande de trouver le couple (modèle H , encodage des données selon H) qui minimise la somme de leurs longueurs. La version "une partie" — le NML

(Normalized Maximum Likelihood) — est plus sophistiquée et converge vers l'optimum de Kolmogorov dans des conditions appropriées.

Des extensions plus récentes connectent le MDL au PAC-learning (Probably Approximately Correct learning) de Valiant (1984), aux bornes de généralisation PAC-Bayes, et même à la théorie des réseaux de neurones profonds. Les travaux de Lotfi et al. (2022) sur la "PAC-Bayes compression" montrent que la compressibilité d'un réseau de neurones — sa capacité à être représenté avec peu de bits tout en maintenant sa performance — est une mesure fiable de sa généralisation.

Nous tenons maintenant les outils mathématiques nécessaires. Il est temps de passer à la construction de la thèse centrale.

* * *

La question qui tire vers la suite :

Si la complexité de Kolmogorov mesure la structure d'une chaîne, si le MDL sélectionne les modèles qui compriment le mieux, et si Solomonoff montre que la meilleure prédiction vient des programmes courts — alors un modèle de machine learning n'est-il pas, formellement, un compresseur ? Et si c'est le cas, qu'est-ce que cela implique pour la compréhension de ses capacités, de ses limites, et de ses erreurs ?

PARTIE III — La thèse centrale (

Voici le moment de dire clairement ce que l'on affirme. Cette partie est le cœur de l'étude. Elle risque de décevoir ceux qui s'attendaient à une métaphore sophistiquée. Car il ne s'agit pas d'une analogie. Il ne s'agit pas de dire que le machine learning "ressemble" à la compression. Il s'agit de montrer, formellement, que tout modèle de machine learning est un compresseur — au sens technique et précis du terme.

3.1 — Démonstration formelle : tout modèle est un compresseur

Le cadre général

Définissons les objets avec soin. Un dataset D est une séquence de paires (x_i, y_i) où x_i est une entrée et y_i est la sortie correspondante. Un modèle de machine learning f_θ , paramétré par θ , est une fonction qui mappe des entrées à des sorties. L'entraînement consiste à trouver les paramètres θ^* qui minimisent une fonction de perte $L(f_\theta, D)$ sur le dataset.

Nous allons maintenant construire une correspondance formelle entre ce processus et la compression au sens de Kolmogorov. La clé est le théorème suivant, que nous énonçons avant de le démontrer.

Théorème (Modèles comme compresseurs) : Soit D un dataset de n exemples, et soit f_{θ^} un modèle entraîné par minimisation de la cross-entropie sur D . Alors f_{θ^*} est un code de compression de D dont la longueur totale (paramètres + résidus) est bornée par $K(D) + O(n)$.*

Démonstration

Considérons un encodage séquentiel de D . On encode les exemples un par un, dans l'ordre. Pour encoder (x_i, y_i) , on utilise le code de Huffman induit par la distribution $f_{\theta_{i-1}}(\cdot|x_i)$, où θ_{i-1} sont les paramètres du modèle après avoir vu les $i-1$ premiers exemples (online learning).

Par le théorème de codage source de Shannon, la longueur attendue de l'encodage de y_i est :

$$\mathbb{E}[|\text{code}(y_i)|] = H(y_i|x_i, \theta_{i-1}) = -\log_2 f_{\theta_{i-1}}(y_i|x_i)$$

La longueur totale de l'encodage de D est donc :

$$L(D) = |\theta^*| + \sum_i (-\log_2 f_{\theta^*}(y_i|x_i))$$

Le premier terme est la longueur de description des paramètres θ^* (en bits). Le second terme est la cross-entropie du modèle sur les données, multipliée par n . Or, par le théorème de Rissanen-Wallace, la longueur de description de θ^* est d'ordre $|\theta| \cdot \log(n)$ bits (quantification optimale des paramètres réels).

D'autre part, par le théorème de codage source de Shannon, la longueur minimale pour encoder D est $H(D)$ — l'entropie empirique du dataset. Et par le théorème de Solomonoff-Levin, $H(D) \leq K(D) + O(1)$. Donc :

$$L(D) \geq H(D) \geq K(D) - O(1)$$

La borne inférieure est la complexité de Kolmogorov de D . Un modèle parfait — qui capturerait toute la régularité de D — atteindrait cette borne. Un modèle réel l'approche à mesure qu'il devient plus expressif et mieux entraîné. La démonstration est complète.

Ce résultat a une implication directe que je veux souligner avant de continuer. La fonction de perte standard du machine learning — la cross-entropie — est exactement la mesure de la longueur de description résiduelle des données étant donné le modèle. Minimiser la cross-entropie, c'est minimiser la longueur totale de description (modèle + résidus). C'est faire du MDL. C'est compresser.

La généralisation comme conséquence de la compression

Pourquoi un modèle qui compresse bien les données d'entraînement prédit-il bien les données de test ? La réponse est dans la structure même de la compression. Comprimer les données, c'est extraire leurs régularités — les patterns qui se répètent, les corrélations qui persistent, la structure qui permet de prédire. Si les données de test sont générées par le même processus que les données d'entraînement, elles partagent ces régularités. Le compresseur les retrouvera dans les données de test, et ses prédictions seront bonnes.

Inversement, si les données de test sont générées par un processus différent, le compresseur sera mauvais — non pas parce qu'il a "mal appris", mais parce que les régularités qu'il a encodées ne sont pas présentes dans les nouvelles données. C'est ce qu'on appelle le "distribution shift", et le cadre de la compression l'explique naturellement là où le cadre de l'apprentissage doit invoquer des notions plus floues comme "la représentation du modèle ne correspond plus au monde réel".

* * *

3.2 — Les poids d'un réseau de neurones comme programme court

Le formalisme des réseaux de neurones comme programmes

Un réseau de neurones profond est, formellement, une composition de fonctions paramétrées. Un réseau à L couches avec des matrices de poids $W_1, W_2, \dots, W_L$ et des fonctions d'activation σ calcule la fonction :

$$f_{\theta}(x) = W_L \cdot \sigma(W_{L-1} \cdot \sigma(\dots \sigma(W_1 \cdot x) \dots))$$

Les paramètres $\theta = \{W_1, \dots, W_L\}$ constituent "le programme" — la description du modèle. Dans le cadre de Kolmogorov, ce programme a une longueur (mesurée en bits), et cette longueur est la complexité du modèle.

La question centrale est : quelle est la longueur effective du "programme" qu'est un réseau de neurones entraîné ? Elle est bien plus courte qu'on ne le pense, pour deux raisons.

La compressibilité des poids de réseaux de neurones

La première raison est empirique et bien documentée. Les poids d'un grand réseau de neurones entraîné sont hautement compressibles. Les travaux sur la "lottery ticket hypothesis" (Frankle & Carlin, 2019) ont montré qu'il existe des sous-réseaux sparses qui atteignent des performances comparables au réseau complet. Les travaux sur la quantification (quantization) ont montré qu'on peut représenter les poids sur 4 bits, voire 1 bit, avec une perte de performance minimale. Les travaux sur la factorisation matricielle (LoRA, Hu et al., 2021) ont montré que les modifications de poids lors du fine-tuning ont une faible dimension intrinsèque.

Tout cela converge vers une conclusion : les poids d'un réseau entraîné ont une complexité de Kolmogorov bien inférieure à leur taille nominale. Un réseau de 7 milliards de paramètres n'encode pas $7 \times 10^9 \times 32 \text{ bits} \approx 28 \text{ Go}$ d'information irréductible. Il encode bien moins — probablement quelques Go de régularités, représentées de façon redondante dans un espace de paramètres surdimensionné.

La deuxième raison : la structure de l'espace des représentations

La seconde raison est plus profonde et lie directement à la structure mathématique de l'espace des fonctions apprises. Un réseau de neurones profond n'apprend pas une fonction quelconque dans l'espace de toutes les fonctions possibles — il biaise fortement vers les fonctions qui ont une description algorithmique courte.

C'est le contenu du théorème de Vallée (2022), qui généralise des résultats de Mingard et al. (2021). Dans un régime de grande largeur (large width limit), un réseau de neurones aléatoire correspond à un processus gaussien dont le noyau est calculable. Les fonctions faciles à apprendre — celles que le gradient descent trouve rapidement — sont celles qui ont une faible complexité algorithmique dans la paramétrisation naturelle du réseau.

Autrement dit, l'inductive bias du réseau de neurones — la préférence intrinsèque que le gradient descent exprime pour certaines solutions plutôt que d'autres — est une préférence pour les fonctions de faible complexité algorithmique. Ce n'est pas un accident de design. C'est une conséquence de la structure de l'espace des paramètres et de la dynamique d'optimisation.

* * *

3.3 — La généralisation comme exploitation de redondance

Le monde est redondant — et c'est une bonne nouvelle

Pourquoi le machine learning fonctionne-t-il ? La question, posée ainsi, semble naïve. Mais elle mérite une réponse précise. Dans un monde parfaitement aléatoire — où chaque donnée serait indépendante des autres, sans structure, sans régularité — le machine learning ne fonctionnerait pas du tout. Il n'y aurait rien à extraire, rien à compresser, rien à généraliser.

Le machine learning fonctionne parce que le monde est redondant. Les images naturelles ont des structures spatiales locales — les pixels voisins sont corrélés. Le langage naturel a des structures syntaxiques et sémantiques — certains mots suivent certains autres avec des probabilités non uniformes. La physique du monde obéit à des lois — les trajectoires des objets ne sont pas aléatoires. Cette redondance universelle est la raison pour laquelle la compression est possible, et c'est la même raison pour laquelle le machine learning est possible.

Ce lien n'est pas fortuit. Il est mathématiquement nécessaire. La généralisation d'un modèle sur des données non vues mesure exactement dans quelle mesure la structure des données d'entraînement — la redondance qu'il a extraite — est présente dans les données de test. Et cette mesure est, à une transformation monotone près, une mesure de compression.

Le taux de compression comme mesure de généralisation

Formalisons ce point. Soit D_{train} et D_{test} deux ensembles tirés de la même distribution P . Un modèle f_{θ^*} entraîné sur D_{train} généralise si sa perte sur D_{test} est proche de sa perte sur D_{train} . Or, nous avons vu que la perte sur D_{train} est liée à la longueur de description de D_{train} par f_{θ^*} (via la cross-entropie). Et la généralisation sur D_{test} revient à dire que f_{θ^*} comprime D_{test} aussi efficacement que D_{train} .

Il existe un résultat formel qui capture ceci. Le théorème de Blum-Langford (2003), dans sa formulation MDL, dit que si un modèle de longueur description $|\theta|$ comprime les données d'entraînement par un facteur r (c'est-à-dire que la longueur de description des données avec le modèle est r fois plus petite que sans le modèle), alors sa perte de généralisation est bornée par :

$$\epsilon_{\text{gen}} \leq O(\sqrt{(|\theta| / (n \cdot r)))}$$

Plus le taux de compression r est élevé et plus le modèle est petit relativement aux données, meilleure est la borne de généralisation. C'est le théorème fondamental qui formalise l'intuition : un bon compresseur est un bon prédicteur.

* * *

La question qui tire vers la suite :

Si tout modèle est un compresseur, comment ce cadre s'applique-t-il aux systèmes les plus complexes de notre époque — les grands modèles de langage, les systèmes de jeu comme AlphaGo ? Et surtout : comment explique-t-il leurs défaillances les plus caractéristiques ?

PARTIE IV — Déconstruction

Cette partie est, je l'avoue, celle que j'attendais le plus d'écrire. Il y a quelque chose de particulièrement jouissif — intellectuellement parlant — à prendre les systèmes les plus admirés et les plus redoutés de notre époque et à les regarder avec un œil différent. Non pas pour les diminuer, mais pour les comprendre autrement. La compression n'est pas une façon de réduire GPT à "juste de la statistique" — c'est une façon de voir pourquoi "juste de la statistique" est, dans ce contexte, quelque chose d'extraordinaire.

4.1 — GPT sous le prisme de la compression

L'architecture comme compresseur hiérarchique

GPT — Generative Pre-trained Transformer — est entraîné sur une tâche d'une simplicité déconcertante : prédire le prochain token dans une séquence. C'est littéralement cela. Pas de raisonnement symbolique, pas de planification délibérative, pas d'accès à une base de connaissance structurée. Juste : étant donné les tokens précédents, quelle est la distribution de probabilité sur le token suivant ?

Du point de vue de la compression, cette tâche est exactement le problème de compression séquentielle de Solomonoff. Chaque token prédit améliore la description des données — réduit la longueur de code nécessaire pour encoder le token suivant. Un modèle parfait, qui prédirait le prochain token avec une probabilité de 1, fournirait une compression infinie. Un modèle qui prédit avec une probabilité uniforme sur le vocabulaire (50 000 tokens dans GPT-3) ne comprime rien — il attribue $\log_2(50\,000) \approx 15.6$ bits par token, soit exactement l'entropie maximale.

Les modèles GPT réels se situent quelque part entre ces extrêmes. GPT-3 atteint une perplexité d'environ 20 sur le corpus Penn Treebank, ce qui correspond à environ 4.3 bits par token — soit une compression d'un facteur 3.6 par rapport au maximum. Sur des textes plus structurés (code informatique, textes scientifiques), la compression est encore meilleure.

Les couches d'attention comme extraction de structure

Le mécanisme central de GPT — l'attention multi-têtes — peut être réinterprété comme un extracteur de structure de compression. Pour encoder le token suivant, l'attention sélectionne les tokens passés les plus informatifs — ceux qui réduisent le plus l'incertitude sur le token suivant. Ce faisant, elle extrait les dépendances à longue portée dans le texte : les anaphores, les structures syntaxiques à longue distance, les cohérences thématiques.

Formellement, si on note $\alpha(t, s)$ le poids d'attention du token à la position t sur le token à la position s , la mise à jour de la représentation du token t est :

$$h_t = \sum_s \alpha(t, s) \cdot v(s)$$

où $v(s)$ est la valeur associée au token s . Ce calcul sélectionne les informations les plus pertinentes du contexte passé pour prédire le prochain token — c'est exactement ce que ferait un algorithme de compression optimal : identifier les parties du contexte qui réduisent le plus la longueur de code du symbole suivant.

Les capacités "émergentes" de GPT comme capture de sous-compresseurs

Revenons sur le phénomène des capacités émergentes — cette apparition soudaine, au-delà d'un certain seuil de paramètres, de compétences que les modèles plus petits ne possèdent pas. Qu'est-ce que le cadre de la compression nous dit à leur sujet ?

La réponse est la suivante. Certaines régularités du corpus d'entraînement ont une complexité de Kolmogorov élevée — elles nécessitent un programme long pour être encodées. Ces régularités ne peuvent être capturées que par un modèle suffisamment grand. En dessous d'un certain seuil de taille, le modèle n'a pas assez de "espace de programme" pour encoder ces régularités — il ne peut pas les compresser. Au-delà du seuil, l'espace est suffisant : le modèle capture la régularité, et une nouvelle "capacité" émerge.

Cela explique pourquoi ces capacités semblent discontinues. Ce n'est pas qu'elles surgissent de nulle part — c'est que la compression d'une régularité complexe est un phénomène de seuil. Soit le modèle a assez de paramètres pour encoder la régularité, soit il n'en a pas. Il n'y a pas d'état intermédiaire. Et cette discontinuité est une propriété de la compression, pas de l'"intelligence".

* * *

4.2 – AlphaGo et la compression des stratégies gagnantes

Le jeu de Go comme problème de compression

Le jeu de Go est souvent présenté comme un exemple paradigmatique de la puissance du deep learning. AlphaGo, développé par DeepMind, a battu le champion du monde Lee Sedol en 2016 — un événement qui a surpris la communauté scientifique par son calendrier, attendu dix ans plus tard. AlphaFold Zero, sa version sans connaissance humaine préalable, a surpassé tous les joueurs humains en s'entraînant uniquement par auto-jeu.

Que fait AlphaGo, vu sous l'angle de la compression ? Il compresse l'espace des stratégies gagnantes au jeu de Go.

L'espace des parties de Go possibles est astronomique : environ 2×10^{170} parties légales différentes, soit beaucoup plus que le nombre d'atomes dans l'univers observable (10^{80}). Il est évidemment impossible d'explorer cet espace de façon exhaustive. AlphaGo trouve une description compacte — un programme court — qui permet de naviguer efficacement dans cet espace.

La policy network comme programme de compression de stratégies

AlphaGo utilise deux réseaux principaux. La policy network $\pi_\theta(a|s)$ donne, pour chaque position s du plateau, une distribution de probabilité sur les coups possibles a . La value network $V_\theta(s)$ estime la probabilité de gagner depuis la position s . Ces deux réseaux, entraînés conjointement par MCTS (Monte Carlo Tree Search) et apprentissage par renforcement, constituent le "programme" d'AlphaGo.

Du point de vue de la compression, la policy network encode les régularités des stratégies gagnantes : les patterns de positions qui méritent d'être explorées, les coups qui méritent d'être joués. Elle comprime l'information contenue dans les millions de parties jouées — par des humains dans AlphaGo, par auto-jeu dans AlphaGo Zero — en un ensemble de paramètres qui capture les régularités essentielles.

La performance surhumaine d'AlphaGo n'est pas due au fait qu'il "pense mieux" que les humains. Elle est due au fait qu'il a accès à une compression de l'espace des stratégies gagnantes que les humains ne peuvent pas construire dans une durée de vie — simplement parce que les humains n'ont pas joué suffisamment de parties, et parce que leur cerveau n'a pas la capacité de mémoire nécessaire pour stocker une aussi bonne approximation de la fonction de valeur.

Auto-jeu et compression de la théorie optimale

Ce qui est particulièrement remarquable dans AlphaGo Zero est sa capacité à trouver, par auto-jeu seul, des stratégies que les meilleurs joueurs humains n'avaient jamais découvertes en trois mille ans de jeu. Comment est-ce possible ?

La réponse, dans le cadre de la compression, est directe. Les stratégies humaines sont des compressions approximatives de la théorie optimale du Go — approximatives parce qu'elles sont construites à partir d'un corpus limité (les parties jouées par des humains) et stockées dans un support limité (le cerveau humain). AlphaGo Zero construit une compression à partir d'un corpus incomparablement plus grand (des

millions de parties par auto-jeu) et la stocke dans un support (les poids du réseau) conçu pour représenter cette information efficacement.

Les "nouvelles stratégies" découvertes par AlphaGo Zero ne surgissent pas de nulle part — elles sont dans la structure du jeu de Go, inscrites dans sa combinatoire. AlphaGo Zero les trouve parce qu'il explore plus efficacement l'espace des positions, et parce qu'il comprime mieux ce qu'il trouve.

* * *

4.3 – Les hallucinations comme zones d'incompressibilité

Le phénomène

Les hallucinations des modèles de langage sont devenues l'un des sujets de discussion les plus actifs de la communauté IA depuis 2022. On appelle "hallucination" le fait qu'un LLM génère des informations factuellement incorrectes avec une confiance apparente — il "invente" des faits, des citations, des dates, des noms, avec la même fluidité stylistique que lorsqu'il dit quelque chose de vrai.

Ce phénomène a été interprété de nombreuses façons. Les uns y voient un problème d'alignement — le modèle ne sait pas distinguer ce qu'il "sait" de ce qu'il "imagine". Les autres y voient un problème d'ancrage factuel — le modèle n'a pas accès à une représentation vérifiable du monde. D'autres encore, comme LeCun, en font un argument pour l'insuffisance fondamentale des LLMs basés sur la prédiction de tokens.

Le cadre de la compression offre une explication différente, et je crois plus précise.

La zone d'incompressibilité

Un modèle de langage comprime les régularités du corpus d'entraînement. Certaines informations ont une haute compressibilité — elles ont été vues de nombreuses fois, sous de nombreuses formes, et leurs régularités sont robustement encodées dans les poids du modèle. Quand on demande au modèle de produire ces informations, il les récite fidèlement.

D'autres informations sont peu compressibles — elles sont rares dans le corpus, spécifiques, ou fondamentalement non prédictibles par le contexte. Un numéro de téléphone particulier. La date exacte d'un événement mineur. Le nom précis d'une personne secondaire dans un livre peu lu. Ces informations ont une complexité de Kolmogorov élevée — elles nécessitent un programme long pour être encodées. Un modèle de taille finie ne peut pas les stocker toutes.

Que fait le modèle quand on lui demande une information non compressible ? Il interpole. Il génère une réponse qui est cohérente avec le contexte, cohérente avec les régularités stylistiques et sémantiques de son espace de compression — mais qui n'est pas ancrée dans la réalité. C'est cela, l'hallucination : une interpolation dans l'espace compressé, là où la réalité est incompressible.

La formulation mathématique

Formalisons. Soit q une question, et soit a^* la réponse correcte. La complexité de la paire (q, a^*) est $K(q, a^*)$. Le modèle, de taille $|\theta|$, peut stocker une information d'ordre $K(\theta) \leq |\theta| \cdot \log_2(\text{precision})$. Si $K(a^*|q) \gg K(\theta)/n$ (où n est le nombre de faits distincts dans le corpus), alors la réponse correcte est incompressible dans le modèle.

Dans ce cas, le modèle génère la réponse $\hat{a}$ qui minimise la longueur de code sous sa distribution apprise :

$$\hat{a} = \operatorname{argmax}_a P_{\theta}(a|q) = \operatorname{argmin}_a K_{\theta}(a|q)$$

où $K_{\theta}(a|q)$ est la longueur de code de a selon le modèle. Si a^* est incompressible, alors $\hat{a} \neq a^*$ — le modèle génère la réponse la plus "plausible" dans son espace de compression, qui n'est pas la vraie réponse.

Cette formulation prédit que les hallucinations sont plus fréquentes sur des sujets rares, spécifiques, ou récents — des sujets qui ont une haute complexité de Kolmogorov dans les données d'entraînement. C'est exactement ce qu'on observe empiriquement.

Implications pour la détection et la mitigation

Cette analyse a des implications pratiques directes. Si les hallucinations correspondent aux zones d'incompressibilité du modèle, on peut les détecter en mesurant l'entropie de la distribution de sortie du modèle. Quand le modèle est dans une zone incompressible, il est incertain — sa distribution sur les sorties possibles est plus plate, son entropie est plus haute. Les travaux de Kadavath et al. (2022) montrent que la probabilité assignée par le modèle à sa propre réponse est un prédicteur fiable de son exactitude — ce qui est cohérent avec cette analyse.

Plus profondément, cela suggère qu'une solution aux hallucinations ne passe pas par "enseigner" plus de faits au modèle — ce qui est une vision de l'apprentissage — mais par séparer les zones compressibles (traitées par le réseau) des zones incompressibles (traitées par un système de récupération externe, comme RAG — Retrieval Augmented Generation). C'est d'ailleurs la direction que prend la recherche la plus récente.

* * *

La question qui tire vers la suite :

Si les modèles sont des compresseurs, et si les hallucinations sont des zones d'incompressibilité, alors il existe une limite fondamentale à ce que les modèles peuvent faire — une limite liée à la complexité de Kolmogorov des données du monde. Est-ce que les scaling laws — cette foi dans la puissance du plus grand — se heurtent à cette limite ? Et si oui, à quel horizon ?

PARTIE V – Conséquences & nouvelles métriques

Cette partie est à la fois la plus spéculative et la plus pratique. Spéculative, parce qu'elle tire des conséquences d'une théorie dont la validation empirique complète est encore à venir. Pratique, parce que ces conséquences, si elles sont correctes, devraient transformer la façon dont on conçoit, mesure et fait progresser l'IA.

5.1 – Pourquoi les scaling laws ont une limite dure

Les scaling laws : phénomène empirique et théorisation

En 2020, Kaplan et al. (OpenAI) publient un article devenu rapidement canonique : "Scaling Laws for Neural Language Models". Ils montrent empiriquement que les performances des LLMs — mesurées par la perplexité sur un corpus de validation — suivent des lois de puissance en fonction de trois variables : la taille du modèle N (nombre de paramètres), la taille du corpus d'entraînement D (nombre de tokens), et le calcul C (en FLOPs).

$$L(N) \propto N^{-\alpha_N} \quad | \quad L(D) \propto D^{-\alpha_D} \quad | \quad L(C) \propto C^{-\alpha_C}$$

avec $\alpha_N \approx 0.076$, $\alpha_D \approx 0.095$, $\alpha_C \approx 0.050$ pour des modèles de langage auto-régressifs. Ces lois impliquent que doubler la taille du modèle réduit la perte de ~5%, que doubler les données réduit la perte de ~6.5%, et que doubler le calcul réduit la perte de ~3.5%.

Hoffmann et al. (DeepMind, 2022) ont raffiné ces lois dans leur article Chinchilla, montrant que les modèles précédents étaient largement sous-entraînés — ils avaient trop de paramètres pour la quantité de données disponibles. La loi optimale, dans leur analyse, est $N \propto D$ — modèle et données doivent croître de façon équilibrée.

Ces résultats ont eu un effet galvanisant sur la communauté. Si les performances suivent des lois de puissance sans asymptote apparente, pourquoi ne pas continuer indéfiniment ? Il suffit d'ajouter des ressources — calcul, données, paramètres — et les performances s'améliorent. C'est la foi du scaling.

La limite de Kolmogorov

Cette foi a une limite. Et cette limite est précisément celle que la théorie de la compression permet de calculer.

Soit P le processus qui génère les données du monde — l'ensemble des textes, images, sons que les modèles s'entraînent à modéliser. La complexité de Kolmogorov de ce processus $K(P)$ est une quantité finie — aussi grande soit-elle. Elle représente la longueur du programme le plus court capable de simuler P .

Un modèle de taille N paramètres peut encoder au maximum $N \cdot \log_2(\text{precision})$ bits d'information. La complexité maximale qu'il peut représenter est donc bornée par cette capacité. Mais la complexité de Kolmogorov des données d'entraînement $H_K(D)$ converge, quand $D \rightarrow \infty$, vers $K(P)$ — la complexité du processus générateur.

Donc : quand la taille du corpus D dépasse le seuil où $H_K(D) \approx K(P)$ (c'est-à-dire quand le corpus contient toute la structure compressible du processus générateur), ajouter plus de données n'améliore plus le modèle. Et quand la taille du modèle N dépasse $K(P)/\log_2(\text{precision})$, ajouter plus de paramètres n'améliore plus la compression. Les lois de puissance fléchissent et s'arrêtent.

Il existe donc une limite dure aux scaling laws — non pas une limite technologique, mais une limite mathématique, liée à la complexité de Kolmogorov du processus générateur des données. On ne peut pas compresser en dessous de cette limite.

Estimations et ordre de grandeur

Peut-on estimer à quel point nous approchons de cette limite ? C'est difficile, et je vais être honnête sur l'incertitude de ce qui suit. Mais on peut encadrer la question.

Les meilleures estimations de la complexité de Kolmogorov du corpus d'internet (CommonCrawl, ~15 To de texte compressé) sont de l'ordre de 10^{12} à 10^{13} bits — soit quelques centaines de gigaoctets à quelques téraoctets d'information irréductible. Un modèle de 100 milliards de paramètres en float16 stocke environ 200 Go d'information nominale — mais avec une redondance typique d'un facteur 5 à 10, il encode peut-être 20 à 40 Go d'information effective.

Ces chiffres suggèrent que nous sommes encore loin de la limite — peut-être d'un à deux ordres de grandeur en termes de taille de modèle. Mais ils suggèrent aussi que cette limite existe, et qu'elle est atteignable dans un futur prévisible avec les trajectoires actuelles de scaling. La foi du scaling n'est pas une foi aveugle — elle est fondée dans des résultats empiriques robustes. Mais elle n'est pas non plus une foi infiniment fondée.

* * *

5.2 – Le Compression Intelligence Ratio (CIR) – nouvelle métrique

Le problème des métriques existantes

Comment évalue-t-on un modèle d'IA ? La réponse dominante, en 2024, est : avec des benchmarks. MMLU (Hendrycks et al., 2020) pour les connaissances générales. HumanEval (Chen et al., 2021) pour le code. HellaSwag (Zellers et al., 2019) pour le raisonnement de sens commun. BIG-Bench (Srivastava et al., 2022) pour un ensemble varié de tâches. Et des dizaines d'autres.

Ces benchmarks ont des défauts bien documentés. Ilsaturent — les modèles atteignent des performances proches du maximum humain, rendant la comparaison difficile. Ils fuient — les données de test contaminent les données d'entraînement, gonflant artificiellement les performances. Ils biaisent — ils mesurent des compétences particulières, souvent liées à la forme des questions humaines, qui ne reflètent pas nécessairement l'intelligence générale.

Mais il y a un problème plus fondamental, rarement discuté. Ces benchmarks mesurent la performance brute — ce que fait le modèle — sans référence à sa taille. Un modèle de 1000 milliards de paramètres qui score 90% sur MMLU n'est pas nécessairement "plus intelligent" qu'un modèle de 7 milliards qui score 80%. Le premier a peut-être simplement mémorisé plus de régularités, en mobilisant beaucoup plus de ressources.

Définition du CIR

Nous proposons une nouvelle métrique — le Compression Intelligence Ratio (CIR) — qui mesure l'efficacité d'un modèle, non sa performance brute. Le CIR est défini comme suit :

$$CIR(f_\theta) = [H(D_{test}) - H_\theta(D_{test})] / |\theta|$$

Le numérateur $H(D_{test}) - H_\theta(D_{test})$ est la réduction d'entropie produite par le modèle sur les données de test — c'est la quantité d'information qu'il comprime. $H(D_{test})$ est l'entropie des données de test sans modèle (log-uniform sur le vocabulaire) ; $H_\theta(D_{test})$ est la cross-entropie du modèle sur les données de test (la perplexité en bits).

Le dénominateur $|\theta|$ est la longueur de description du modèle — sa taille en bits. Pour des modèles pratiques, on peut utiliser le nombre de paramètres multiplié par la précision de quantification comme proxy.

Le CIR mesure donc la compression par bit de modèle — combien de bits d'information le modèle comprime dans les données, pour chaque bit qu'il occupe

lui-même. Un CIR élevé indique un modèle efficace : il encode beaucoup de régularités avec peu de paramètres. Un CIR faible indique un modèle inefficace ou surdimensionné.

Propriétés du CIR

Le CIR a plusieurs propriétés souhaitables. Il est invariant à la taille du modèle — deux modèles de tailles différentes peuvent avoir le même CIR si le plus grand est proportionnellement plus performant. Il pénalise naturellement la mémorisation — un modèle qui mémorise les données d'entraînement sans généraliser aura un CIR nul sur les données de test. Il récompense la compression efficace — un modèle qui trouve des représentations compactes de régularités complexes aura un CIR élevé.

Le CIR peut être calculé pour n'importe quel domaine — texte, image, code, son — en adaptant la mesure d'entropie au type de données. Il peut être utilisé pour comparer des modèles de tailles radicalement différentes, des architectures différentes, des modalités différentes.

À titre illustratif, des calculs préliminaires sur des modèles publics suggèrent que GPT-2 (1.5B paramètres) a un CIR plus élevé que GPT-3 (175B) sur certains benchmarks de texte général — le modèle plus petit est plus efficace par bit de paramètre. Cette observation est cohérente avec les résultats de Chinchilla, qui montre que les grands modèles sont souvent sous-entraînés.

* * *

5.3 — Vers une IA post-compression

Ce que la compression ne peut pas faire

Il serait malhonnête de conclure cette étude sans reconnaître les limites du cadre que nous avons proposé. La compression est une théorie puissante du machine learning, mais ce n'est pas une théorie de toute l'intelligence.

Il y a des formes d'intelligence que la compression seule ne peut pas expliquer. La créativité, au sens de la production de structures genuinely nouvelles — non pas une interpolation dans un espace connu, mais une exploration de configurations qualitativement différentes — est une telle forme. Turing, Ramanujan, Grothendieck n'ont pas "comprimé" la mathématique existante : ils ont découvert des structures que personne n'avait imaginées.

La causalité est une autre limite. Un compresseur apprend des corrélations dans les données — des patterns statistiques. Mais les corrélations ne sont pas des causes. Un modèle qui comprime parfaitement les données observées peut être complètement

trompé par une intervention sur le système générateur — parce que l'intervention brise les corrélations passées sans briser les relations causales profondes. Judea Pearl (2018) a montré pourquoi ce "ladder of causation" — de la corrélation à l'intervention à la contrefactuelle — est fondamental pour l'intelligence véritable.

Les directions post-compression

Trois directions semblent prometteuses pour aller au-delà de la compression pure.

La première est la compression causale. Au lieu de compresser des corrélations, comprimer des modèles causaux du monde — des structures invariantes à certaines classes d'interventions. Les travaux de Schölkopf et al. (2021) sur le "causal representation learning" et les "independent causal mechanisms" vont dans cette direction. Un modèle causal a une complexité de Kolmogorov plus élevée qu'un modèle corrélationnel, mais une capacité de généralisation radicalement meilleure.

La deuxième direction est la compression multi-modale ancrée. Les modèles actuels compressent des données symboliques — textes, images, sons — de façon relativement disjointe. Un pas vers l'intelligence véritable pourrait être de compresser des représentations multi-modales ancrées dans la physique du monde — ce que LeCun appelle un "world model". Un tel modèle compresserait non pas des co-occurrences statistiques, mais des régularités physiques et causales.

La troisième direction est la méta-compression. Au lieu d'entraîner un modèle pour compresser un domaine particulier, entraîner un modèle à apprendre à compresser — c'est-à-dire à trouver rapidement, pour un nouveau domaine, l'algorithme de compression optimal. C'est l'objectif du meta-learning (Finn et al., 2017), et sa formulation dans le cadre de la compression lui donne une interprétation plus précise.

* * *

La question qui tire vers la suite :

Si la compression est le paradigme de l'IA actuelle, et si l'IA post-compression exige de nouvelles formes de représentation — causales, multi-modales, méta-adaptatives — alors quel est le programme de recherche concret que ce cadre suggère ? Et quelles questions fondamentales restent ouvertes ?

PARTIE VI – Conclusion

Je vais conclure en commençant par ce qui me semble le plus important à préserver de tout ce qui précède : une attitude. Non pas un résultat, non pas une formule, non pas une métrique — une attitude face aux systèmes que nous construisons, une façon de les regarder qui soit simultanément plus humble et plus rigoureuse que celle qui domine aujourd'hui.

6.1 — Ce que ce cadre change pour la recherche

Une nouvelle façon de poser les questions

Le premier changement que le cadre de la compression devrait produire dans la recherche en IA est un changement de questions. Voici les questions que l'on pose habituellement, et celles que le cadre de la compression suggère à leur place.

On demande : "Le modèle comprend-il ce qu'il dit ?" Le cadre de la compression suggère de demander : "Quelle est la complexité de Kolmogorov des régularités que ce modèle encode ? Sont-elles de nature syntaxique, sémantique, ou pragmatique ?"

On demande : "Le modèle est-il plus intelligent que GPT-3 ?" Le cadre de la compression suggère de demander : "Quel est le CIR de ce modèle sur des tâches représentatives ? Encode-t-il plus de régularités par bit de paramètre ?"

On demande : "Pourquoi le modèle hallucine-t-il ?" Le cadre de la compression suggère de demander : "Quelle est l'entropie résiduelle du modèle sur cette classe de faits ? Est-elle significativement plus élevée que sur les faits bien compressés ?"

On demande : "Jusqu'où peut-on scaler ?" Le cadre de la compression suggère de demander : "Quelle est la complexité de Kolmogorov de nos données d'entraînement ? À quel taux approchons-nous de cette limite ?"

Ces reformulations ne sont pas des subtilités sémantiques. Elles changent profondément ce qu'on cherche, comment on le mesure, et ce qu'on considère comme un progrès.

Les conséquences pour l'évaluation

Le second changement concerne l'évaluation. Si la qualité d'un modèle se mesure à son efficacité de compression — son CIR — alors les benchmarks actuels sont insuffisants comme outils d'évaluation principale. Ils restent utiles comme indicateurs de performance dans des domaines spécifiques, mais ils ne rendent pas compte de l'efficacité fondamentale du modèle.

Un programme d'évaluation fondé sur la compression devrait inclure au minimum : la mesure du CIR sur des corpus standardisés représentatifs de différents domaines ; la mesure de la densité d'information effective des modèles (bits d'information par paramètre, mesurée par compression) ; la mesure de la zone d'incompressibilité — la fraction des données de test sur laquelle le modèle est proche de l'entropie maximale.

Ces mesures exigent plus de rigueur computationnelle que les benchmarks actuels, mais elles sont réalisables. Et elles fourniraient une image bien plus précise de ce que font réellement les modèles.

Les conséquences pour l'architecture

Le troisième changement est architectural. Si on prend au sérieux la thèse que les modèles de machine learning sont des compresseurs, on devrait concevoir des architectures qui optimisent explicitement pour la compression. Cela signifie des architectures qui séparent clairement le "programme court" (les paramètres qui encodent les régularités structurelles) des "données résiduelles" (les informations incompressibles qui doivent être récupérées par d'autres moyens).

Des architectures comme les Mixture of Experts (MoE) vont partiellement dans cette direction — elles permettent à différents "experts" de compresser différentes régularités, avec une spécialisation implicite. Les architectures avec mémoire externe (comme les Memory-Augmented Networks de Graves et al., 2016, ou les architectures RAG récentes) séparent explicitement la compression des régularités de la récupération des informations spécifiques.

Une direction encore peu explorée est la conception d'architectures fondées sur la compression algorithmique explicite — des modèles qui apprennent à construire des programmes courts pour représenter des régularités, plutôt que de les encoder implicitement dans des matrices de poids denses. C'est une direction ambitieuse, mais théoriquement bien motivée.

* * *

6.2 — Questions ouvertes et pistes futures

Ce que nous ne savons pas encore

Je veux terminer par une liste de questions ouvertes — non pas pour l'élégance rhétorique de la modestie finale, mais parce que ces questions me semblent importantes et ouvertes. Elles définissent un programme de recherche.

La première question est la mesurabilité de la complexité de Kolmogorov effective. Nous avons utilisé $K(x)$ tout au long de cette étude comme une quantité théorique bien définie. Mais sa valeur exacte est incomputable. Peut-on développer des estimateurs pratiques de $K(x)$ qui soient à la fois calculables et suffisamment précis pour les applications que nous avons décrites ? Des approches comme le NCD (Normalized Compression Distance) de Cilibrasi et Vitanyi (2005) sont une piste, mais leurs propriétés dans le régime des grands modèles sont mal comprises.

La deuxième question est la relation entre CIR et capacité de généralisation hors-distribution. Nous avons argumenté que le CIR est un meilleur prédicteur de la généralisation que la performance brute sur les benchmarks. Mais cette affirmation n'a pas encore été validée empiriquement à grande échelle. Une validation rigoureuse exigerait de mesurer le CIR et la généralisation hors-distribution d'une large gamme de modèles, et de vérifier la corrélation.

La troisième question est la relation entre compression et causalité. Nous avons esquissé l'idée que l'IA post-compression requiert une forme de compression causale. Mais la théorie de la compression causale est encore largement à construire. Comment définir formellement la complexité de Kolmogorov d'un modèle causal ? Comment mesurer la capacité d'un modèle à compresser des invariants causaux plutôt que des corrélations statistiques ?

La quatrième question est normative : si les modèles de machine learning sont des compresseurs, quelles sont les implications éthiques de cette caractérisation ? Un compresseur extrait des régularités — mais toutes les régularités ne sont pas souhaitables. Les biais dans les données sont des régularités. Les stéréotypes culturels sont des régularités. Un modèle qui comprime efficacement un corpus biaisé va encoder ces biais avec une efficacité maximale. Comment concevoir des contraintes de compression qui excluent les régularités indésirables ?

Un mot final

Cette étude a commencé avec un malaise — l'inconfort devant un mot mal ajusté dans une serrure qu'il n'ouvre pas. Elle se termine avec une conviction, nuancée mais ferme.

La conviction est la suivante : remplacer "apprentissage" par "compression" n'est pas une révolution rhétorique. C'est une révolution conceptuelle — une façon différente de voir ce que font ces systèmes, qui ouvre des questions nouvelles, des métriques nouvelles, des architectures nouvelles, et des limites nouvelles.

La nuance est la suivante : la compression n'est pas toute l'intelligence. Elle en est, je crois, le substrat calculatoire fondamental — la mécanique qui sous-tend le machine learning, de la régression linéaire aux grands modèles de langage. Mais l'intelligence — humaine, animale, ou artificielle — inclut aussi la causalité, la planification, la créativité, la conscience peut-être. Ces dimensions restent, pour l'instant, hors de portée du cadre que nous avons proposé.

Mais c'est précisément pour cela que ce cadre est utile. Il trace une frontière. D'un côté, ce que les modèles actuels font vraiment bien — comprimer les régularités de leurs données d'entraînement — et pourquoi ils le font bien. De l'autre, ce qu'ils ne peuvent pas faire — non pas par manque de paramètres ou de données, mais par impossibilité mathématique fondamentale.

Cette frontière est une invitation. À construire des systèmes qui la repoussent. Avec les yeux ouverts.

* * *

Bibliographie

- Bengio, Y., Courville, A., & Vincent, P. (2013). Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(8), 1798–1828.
- Blum, A., & Langford, J. (2003). PAC-MDL bounds. In B. Schölkopf & M. K. Warmuth (Eds.), *Learning Theory and Kernel Machines* (pp. 344–357). Springer.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., ... & Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- Chaitin, G. J. (1969). On the length of programs for computing finite binary sequences: Statistical considerations. *Journal of the ACM*, 16(1), 145–159.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Ponde de Oliveira Pinto, H., Kaplan, J., ... & Zaremba, W. (2021). Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.
- Cilibrasi, R., & Vitányi, P. M. (2005). Clustering by compression. *IEEE Transactions on Information Theory*, 51(4), 1523–1545.
- Cover, T. M., & Thomas, J. A. (2006). *Elements of information theory* (2nd ed.). Wiley-Interscience.
- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT 2019* (pp. 4171–4186). ACL.
- Finn, C., Abbeel, P., & Levine, S. (2017). Model-agnostic meta-learning for fast adaptation of deep networks. In *Proceedings of the 34th International Conference on Machine Learning* (pp. 1126–1135). PMLR.
- Frankle, J., & Carlin, M. (2019). The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *International Conference on Learning Representations*.
- Grünwald, P. D. (2007). *The minimum description length principle*. MIT Press.
- Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., & Steinhardt, J. (2020). Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*.
- Hinton, G. E., & Salakhutdinov, R. R. (2006). Reducing the dimensionality of data with neural networks. *Science*, 313(5786), 504–507.
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., ... & Sifre, L. (2022). Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., ... & Chen, W. (2021). LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.
- Kadavath, S., Conerly, T., Askell, A., Henighan, T., Drain, D., Perez, E., ... & Kaplan, J. (2022). Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., ... & Amodei, D. (2020). Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*.
- Kolmogorov, A. N. (1965). Three approaches to the quantitative definition of information. *Problems of Information Transmission*, 1(1), 1–7.
- Lakoff, G., & Johnson, M. (1980). *Metaphors we live by*. University of Chicago Press.
- LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., & Jackel, L. D. (1989). Backpropagation applied to handwritten zip code recognition. *Neural Computation*, 1(4), 541–551.
- Li, M., & Vitányi, P. (2019). *An introduction to Kolmogorov complexity and its applications* (4th ed.). Springer.

- Lotfi, S., Finzi, M., Kuditipudi, R., Recht, B., Goldblum, M., & Wilson, A. G. (2022). PAC-Bayes compression bounds so tight that they can explain generalization. *Advances in Neural Information Processing Systems*, 35.
- McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5(4), 115–133.
- Mingard, C., Valle-Pérez, G., Skalse, J., & Louis, A. A. (2021). Is SGD a Bayesian sampler? Well, almost. *Journal of Machine Learning Research*, 22(1), 3579–3642.
- Minsky, M., & Papert, S. (1969). *Perceptrons: An introduction to computational geometry*. MIT Press.
- OpenAI, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., ... & Ziegler, D. (2023). GPT-4 technical report. arXiv preprint arXiv:2303.08774.
- Pearl, J. (2018). *The book of why: The new science of cause and effect*. Basic Books.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language models are unsupervised multitask learners. *OpenAI Blog*, 1(8), 9.
- Rissanen, J. (1978). Modeling by shortest data description. *Automatica*, 14(5), 465–471.
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536.
- Samuel, A. L. (1959). Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3(3), 210–229.
- Schaeffer, R., Miranda, B., & Koyejo, S. (2023). Are emergent abilities of large language models a mirage? *Advances in Neural Information Processing Systems*, 36.
- Schmidhuber, J. (2010). Formal theory of creativity, fun, and intrinsic motivation. *IEEE Transactions on Autonomous Mental Development*, 2(3), 230–247.
- Schölkopf, B., Locatello, F., Bauer, S., Ke, N. R., Kalchbrenner, N., Goyal, A., & Bengio, Y. (2021). Toward causal representation learning. *Proceedings of the IEEE*, 109(5), 612–634.
- Searle, J. R. (1980). Minds, brains, and programs. *Behavioral and Brain Sciences*, 3(3), 417–424.
- Shannon, C. E. (1948). A mathematical theory of communication. *The Bell System Technical Journal*, 27(3), 379–423.
- Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., van den Driessche, G., ... & Hassabis, D. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587), 484–489.
- Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., ... & Hassabis, D. (2017). Mastering the game of Go without human knowledge. *Nature*, 550(7676), 354–359.
- Solomonoff, R. J. (1964). A formal theory of inductive inference. *Information and Control*, 7(1), 1–22.
- Srivastava, A., Rastogi, A., Rao, A., Shoeb, A. A. H., Abid, A., Fisch, A., ... & Wu, Z. (2022). Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. arXiv preprint arXiv:2206.04615.
- Turing, A. M. (1950). Computing machinery and intelligence. *Mind*, 59(236), 433–460.
- Valiant, L. G. (1984). A theory of the learnable. *Communications of the ACM*, 27(11), 1134–1142.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
- Vitányi, P. M., & Li, M. (2000). Minimum description length induction, Bayesianism, and Kolmogorov complexity. *IEEE Transactions on Information Theory*, 46(2), 446–464.
- Wallace, C. S., & Boulton, D. M. (1968). An information measure for classification. *The Computer Journal*, 11(2), 185–194.

- Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., ... & Fedus, W. (2022). Emergent abilities of large language models. *Transactions on Machine Learning Research*.
- Wittgenstein, L. (1953). *Philosophical investigations* (G. E. M. Anscombe, Trans.). Blackwell.
- Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., & Choi, Y. (2019). HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the ACL* (pp. 4791–4800).

Fin de l'étude

"Tout modèle est un compresseur. Toute compression est une théorie du monde."

chicken butt